

HTTP Requests

HTTP

- Everyone has heard of HTTP in some capacity, especially if you have used a web browser... which is all of you.
- Hypertext Transfer Protocol
- Typically is on port 80 (normally the default)

HTTP Methods

- There are many different HTTP methods out there
- We will only focus on two, which you most likely have seen in some fashion
 - GET
 - Simply retrieving information
 - POST
 - Simply putting information externally
- HTTP normally has the request and a response

HTTP Requests

- Think of how you put a URL into a browser
- Navigate to <http://www.google.com>
 - That is a HTTP request
- Requests will 'request' data from a given url
 - We can also pass data with a given request:
 - Parameters: normally specify what data is returned (?id=1)
 - Try using /search?q= in Google
 - Body: normally data to be uploaded/downloaded (we'll see this soon)
 - Header: Coordination b/w server and client (pass date, caching mechanism, and even authorization)

HTTP Response Body

- For a given request from the client, the server should have a response
- Most likely this response will be in the form of JSON
 - Although some really old and out-of-date servers still use XML
- Our job as a requester is to:
 - Ensure we passed valid parameters/body/headers (if applicable)
 - Have a valid response code
 - Can parse the JSON data correctly to get the information we need

Status Codes

- A developer should always verify that there is a valid response code
- It varies based on the API you are trying to communicate
- <http://www.restapitutorial.com/httpstatuscodes.html>

Networking Node Modules

- There is the 'http' and 'request' modules (older)
- HTTP
 - Contains support for the raw HTTP protocol (a lot of fancy stuff that you and I don't need to know or worry about)
- Request
 - Uses HTTP and adds a lot of extra stuff to make it easy to consume and use.
 - 'npm install request'

Promised Based Network Requests

- Becoming much more popular in front-end development. Largely popularized due to the way AJAX/JQuery worked
- The two most common are:
 - Fetch API
 - node-fetch package
 - Axios
 - Most widely adopted in industry currently
 - Built on top of XMLHttpRequest so can register onUploadProgress and onDownloadProgress

Requests Using Axios

- Main difference is the “.then” statement
 - This is considered a “promise” that is to be executed later
 - In this case, when the request returns

```
const axios = require('axios');  
  
axios.get('https://www.google.com')  
  .then(resp => {  
    console.log(resp.data);  
  });
```

Anatomy of a Request function (GET)

```
var axios = require('axios');

axios.get('http://www.google.com')
  .then(function(response) {
    if (response.status == 200) {
      console.log(response.data)
    }
  })
  .catch(function (error) {
    console.log(error);
  });
```

simpleRequest.js

```
(1,{t:66}); href="https://www.google.com/intl/en/options/" class="gbmt">Even more &raquo;</a></li></ol></div><div class="gbsbt"></div><div class="gbsbb"></div></div></li></ol></div><div id="gbg"><h2 class="gbxx">Account Options</h2><span class="gbtcb"></span><ol class="gbtc"><li class="gbt"><a target="_top" href="https://accounts.google.com/ServiceLogin?hl=en&passive=true&continue=http://www.google.com/" onclick="gbar.logger.il(9,{l:'i'})" id="gb_70" class="gbgt"><span class="gbtb2"></span><span id="gbgs4" class="gbts"><span id="gbi4s">Sign in</span></span></a></li><li class="gbt" class="gbts"><span class="gbts"></span></li><li class="gbt"><a class="gbgt" id="gbg5" href="http://www.google.com/preferences?hl=en" title="Options" onclick="gbar.tg(event,this)" ar id="haspopup=true aria-owns="gbd5"><span class="gbtb2"></span><span id="gbgs5" class="gbts"><span id="gbi5"></span></span></a><div class="gbm" id="gbd5" aria-owns="gbg5"><div class="gbmc"><ol id="gbom" class="gbmc"><li class="gbkc" class="gbmt"><a class="gbmt" href="/preferences?hl=en">Search settings</a></li><li class="gbmtc"><div class="gbmt" class="gbmh"></div></li><li class="gbkp" class="gbmt"><a class="gbmt" href="http://www.google.com/history/optout?hl=en">Web History</a></li></ol></div><div class="gbx3"></div><div id="gbx4"></div><script>window.gbar&gbgbar.e.lp&gbgbar.elp()</script></div></div><center><br clear="all" id="lgpd"><div id="lga"><br><br></div><form action="/search" name="f"><table cellpadding="0" cellspacing="0"><tr valign="top"><td width="25%">&nbsp;</td><td align="center" nowrap=""><input name="ie" value="ISO-8859-1" type="hidden"><input value="en" name="hl" type="hidden"><input name="source" type="hidden" value="hp"><input name="biw" type="hidden"><input name="bih" type="hidden"><div class="ds" style="height:32px;margin:4px 0"><input style="color:#000;margin:0;padding:5px 8px 0 6px;vertical-align:top" autocomplete="off" class="lst" value="" title="Google Search" maxlength="2048" name="q" size="57"></div><br style="line-height:0"><span class="ds"><span class="lsbb"><input class="lsb" value="Google Search" name="btnG" type="submit"></span></span><span class="ds"><span class="lsbb"><input class="lsb" value="I'm Feeling Lucky" name="btnI" onclick="if(this.form.q.value)this.checked=1; else top.location='/doodles/'" type="submit"></span></span></td><td class="fl sbic" align="left" nowrap="" width="25%"><a href="/advanced_search?hl=en&authuser=0">Advanced search</a></td></tr></table><input id="gbv" name="gbv" type="hidden" value="1"><script nonce="uBTtkbJgFcud+VCD0QrdIQ=">(function(){var a,b="1";if(document&document.getElementById){if("undefined"!=typeof XMLHttpRequest){for(c=0;d=e[c++];try{new XMLHttpRequest(d,b="2")}catch(h)){a=b;if("2"===a&1=location.search.indexOf("&gbv=2")){var f=google.gbu.gv-document.getElementById("gbv");g&(g.value=a);f&window.setTimeout(function(){location.href=f},0);}}.call(th is)}</script></form><div id="gac_scont"></div><div style="font-size:83%;min-height:3.5em"><br></div><span id="footer"><div style="font-size:10pt"><div style="margin:19px auto;text-align:center" id="fil"><a href="http://www.google.com/ads/">Advertising</a><a href="http://www.google.com/services/">Business Solutions</a><a href="https://plus.google.com/116899029375914044550" rel="texta er">Google</a><a href="http://www.google.com/about.html">About Google</a></div><div><p style="color:#767676;font-size:8pt">©2018 - <a href="http://www.google.com/policies/privacy">Privacy</a> - <a href="http://www.google.com/policies/terms">Terms</a></p></div><div><script nonce="uBTtkbJgFcud+VCD0QrdIQ=">(function(){window.google.cdo={height:0,width:0};(function(){var a=window.innerWidth,b=window.innerHeight;if(!a||!b){var c=window.document,d="CSS1Compat"===c.compatMode?c.documentElement:c.body;a=d.clientWidth;b=d.clientHeight}&&google.cdo.width||b=google.cdo.height}&&google.log("",""/client_2047&atyp=i&biw="+a&bih="+b&et="+google.kEID));.call(this)}</script><div id="xjsd"></div><div id="xjsi"><script nonce="uBTtkbJgFcud+VCD0QrdIQ=">(function(){function c(b){window.setTimeout(function(){var a=document.createElement("script");a.src=b;google.timers&google.timers.load.t&google.tick("load",{gen204:"xjsls",clearcut:31});document.getElementById("xjsd").appendChild(a)});google.d.ljp=function(b,a){google.xjsu=b;c(a)};google.d.ljp=c;}.call(this);if(!google.xjs){window._=window._||{};window._.dumpException=function(e){throw e};window._.f.installCss=function(c){google.d.ljp('/xjs/_js/kx3dxjs.hp.en.US.FmYwL9.C198.0/m/x3dsb_he,d/am/x3dVdAm/r/t/x3dj/d/x3d1/t/x3dzcms/rs/x3dACT900GaQxCP85gup_1WM80EpxI8872A')>google.xjs=1}>google.pmc={s:"sb_he":{,"agen":true,"cgen":true,"client":true,"heirloom-hp":true,"dh":true,"dhqt":true,"ds":true,"fql":true,"fl":true,"host":"google.com","isb":28,"jsonp":true,"msgs":{"cibl":"","Clear Search"},"dym":"Did you mean:","lcky":"Tu0026f39;m Feeling Lucky"},"lml":"Learn more","osk":"Input tools"},"psrcr":"This search was removed from your Tu003Ca href="/history/">History</a>Tu003C/a>Tu003E","psrl":"Remove","sbtl":"Search by image","srch":"Google Search"},"nds":"true","ovr":"","pq":"","refpd":true,"rfs":"","sblp":24,"sbpr":24,"scd":10,"sce":5,"stok":"YvTaTnTmhVSfCaUyIuFiYeAo9Q"},"d":{"ZI/VVQ":{},"U582ig":{},"YFCs/g":{}};google.x=null,function(){}>(function(){var r=[];google.plm(r)}</script></div></body></html></pre>
```

GET With Parameters

```
var axios = require('axios');

axios.get('https://jsonplaceholder.typicode.com/posts?id=1')
  .then(function(response) {
    if (response.status == 200) {
      console.log(response.data)
    }
  }).catch(function (error) {
    console.log(error);
  });
```

requestParamsUrl.js

```
Jals-MacBook-Pro:Desktop jalirani$ node requestParamsUrl.js
[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
  }
]
Jals-MacBook-Pro:Desktop jalirani$
```

GET With Parameters (cont.)

- Note that the parameters can be passed through the url since it is a GET
- We can also pass the parameters using the key 'params' as an object

Get Parameters in Object

```
var axios = require('axios');

axios.get('https://jsonplaceholder.typicode.com/posts', {
  params: {
    userId: 1
  }
}).then(function(response) {
  console.log(response.status)
  console.log(response.data)
}).catch(function (error) {
  console.log(error);
});
```

requestParamsObj.js

```
requestParamsObj.js requestParamsUrl.js requestParamsObj.js
Jals-MacBook-Pro:Desktop jalirani$ node requestParamsUrl.js
[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
  }
]
Jals-MacBook-Pro:Desktop jalirani$ node requestParamsObj.js
[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
  },
  {
    "userId": 1,
    "id": 2,
    "title": "qui est esse",
    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
  },
  {
    "userId": 1,
    "id": 3,
    "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
    "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et labore et velit aut"
  },
  {
    "userId": 1,
    "id": 4,
    "title": "eum et est occaecati",
    "body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic commodi nesciunt rem tenetur doloremque ipsam iure\nquis sunt voluptatem rerum illo velit"
  },
]
```

JSON.parse Function

- Just like we used `JSON.stringify()` to turn a JSON object into a String
- When we get a response from an API, we use `JSON.parse` to turn a String into a JSON object.
 - Example: `JSON.parse(body);`
 - https://www.w3schools.com/js/js_json_parse.asp

oAuth and Tokens

- oAuth is the standard for open authorization on the internet
- This is a way for a user to verify that it is him/her making an HTTP request
- Allows an end user's account information to be used by third-party services, such as Facebook, without exposing the user's password
- Essentially it is a token provided by the API to ensure you are the correct user
 - If a service like oAuth did not exist, anyone could GET and POST

Spotify GET Example

- spotifyShow.js
- <https://developer.spotify.com/documentation/web-api>
 - List of all endpoints available
- <https://developer.spotify.com/dashboard>
 - Gets API keys

POST Example

```
var axios = require('axios');

axios.post('https://jsonplaceholder.typicode.com/posts', {
  title: 'Test title',
  body: 'Test body',
  userId: 11
}).then(function (response) {
  console.log(response.data)
  console.log(response.status)
}).catch(function (error) {
  console.log(error);
});
```

Group Requests

- Can make multiple requests with different verbs
- Returns **in order** from when it was called

```
axios.all([
  axios.get('https://jsonplaceholder.typicode.com/posts?userId=1'),
  axios.get('https://jsonplaceholder.typicode.com/posts?userId=2')
])
.then(responseArr => {
  console.log('Posts for id=1: ', responseArr[0].data);
  console.log('Posts for id=2:', responseArr[1].data);
});
```

Further Exploration

- Multipart/form-data
 - [onUploadProgress](#)
- [Cancel Request](#)
 - Common in search bar requests or navigation to a page

Test API

<https://jsonplaceholder.typicode.com/>